कोडनीती AI (CodeNiti AI) — Final Year Major Project Presentation

Department of Electronics & Computer Engineering
CSMSS Chh. Shahu College of Engineering, Chhatrapati Sambhajinagar
Academic Year: 2026–27

Slide 1: Title Slide

कोडनीती AI (CodeNiti AI)

AI-Powered Code Intelligence Platform for Automated Optimization, Complexity Analysis & Security Auditing

- **Department:** Electronics & Computer Engineering
- **Institute:** CSMSS Chh. Shahu College of Engineering, Chhatrapati Sambhajinagar
- **Project Guide:** Prof. J. N. Mohite

Roll No.	Enrollment / PRN	Student Name
EC4145	23025331844046	**Dhuppe Harshad Madhav**
EC4121	23025331844021	**Shinde Saurabh Bhagaji**
EC4168	24025331844502	**Kale Pradeep Dyaneshwar**

> **Speaker Note:**
> "Respected Guide Prof. J. N. Mohite sir and panel members, good morning. We are presenting our final year major project, CodeNiti AI — an engineering platform designed to eliminate manual code review bottlenecks by integrating deterministic AST-based developer tools with generative AI reasoning."

Slide 2: Problem Statement

Current Developer Bottlenecks

1. **High Manual Code Review Overhead:**
   - Engineering teams spend **25%–30% of their working hours** manually checking boilerplate syntax, formatting inconsistencies, and recurring algorithmic anti-patterns.
2. **Inaccurate Complexity & Algorithmic Analysis:**
   - Estimating Big-O asymptotic runtime ($O(N)$, $O(N \log N)$, $O(N^2)$) and auxiliary space across nested branches and recursive calls is frequently misjudged.
3. **Overlooked Security Vulnerabilities:**
   - Critical injection risks (e.g., `subprocess.run(shell=True)`, SQL injection, insecure deserialization) pass silently into production pipelines.
4. **Deficiencies of Generic AI Wrappers:**

- **Cost & Latency Waste:** Repeated queries re-run the cloud LLM, wasting tokens and incurring 5–10 second network delays.
- **Hallucinated Security:** Standard LLMs hallucinate false-positive security proofs.
- **Privacy Exposure:** Uncontrolled transmission of sensitive proprietary code to third-party cloud vendors.

> **Speaker Note:**
> "Pure AI wrappers are not production-ready for developers. They cost tokens on duplicate runs, hallucinate security audits, and risk code privacy. CodeNiti AI solves this through a hybrid deterministic-and-generative engineering approach."

Slide 3: Project Objectives

Core Goals of CodeNiti AI

- **1. Build a Hybrid Code Intelligence Platform:**
  - Fuse cloud LLM semantic reasoning (**Google Gemini 2.5 Flash**) with **deterministic local offline engines** (Ruff, Bandit, Difflib) to ensure mathematically reproducible results where needed.
- **2. Zero-Execution Security Guarantee:**
  - Inspect code via **Abstract Syntax Trees (AST)** and static text diffs without ever executing untrusted user code on the host operating system.
- **3. Sub-Millisecond Multi-Level Caching:**
  - Implement an **L1 Memory LRU + L2 SQLite WAL** cache keyed by SHA-256 fingerprints to reduce duplicate AI call latency by **>99%** (< 5 ms).
- **4. Enterprise-Grade Authentication & Multi-Tenancy:**
  - Secure developer workspaces using **PBKDF2-HMAC-SHA256** ($600{,}000$ iterations), opaque revocable tokens, and strict per-account data isolation.
- **5. Strict Zero-Trust Client Architecture:**
  - Build a zero-dependency Vanilla JS UI protected by **Content Security Policy (CSP)** and anti-CSRF headers, rendering DOM nodes without `innerHTML` XSS vectors.

> **Speaker Note:**
> "Our primary objective is safety and speed: developers get instant formatting, deterministic AST security scanning, and AI algorithmic insights without host execution risks or token waste."

Slide 4: Literature Survey & Related Work

Comparative Technological Analysis

Feature / Dimension	Generic LLM Chat (ChatGPT / Claude)	Traditional Linters (SonarQube / ESLint)	**CodeNiti AI (Our Proposed System)**
Architecture Model	100% Cloud Generative	100% Static Rule Engine	**Hybrid (Deterministic AST + GenAI)**
Offline Execution	❌ Requires Active Internet	✅ Local Runtime	✅ **Offline Local Tools (Ruff / Bandit / Diff)**

Feature / Dimension	Generic LLM Chat (ChatGPT / Claude)	Traditional Linters (SonarQube / ESLint)	CodeNiti AI (Our Proposed System)
Response Caching	❌ Billed on duplicate calls	❌ Not applicable	✅ **2-Level SHA-256 Fingerprint Cache**
Security Auditing	⚠️ Prone to hallucinations	⚠️ Rule check only, no fix hints	✅ **Bandit AST Static Scan + Remediation Guide**
Complexity Analysis	⚠️ Text explanation only	⚠️ Basic cyclomatic number	✅ **Asymptotic Time & Space Analysis with Proofs**
Arbitrary Code Risk	⚠️ Unbounded execution prompts	❌ None	✅ **AST parsing only (Zero code execution)**
Privacy Mode	❌ Prompts saved by provider	✅ On-premise	✅ **Opt-In History Retention**

Key Takeaway from Survey: Existing systems are either rigid static checkers or pure generative chatbots. CodeNiti AI bridges this gap with a hybrid layered architecture.

> **Speaker Note:**
> "As seen in our comparative analysis, SonarQube lacks contextual generative explanations, while ChatGPT wastes tokens and hallucinates security audits. CodeNiti AI combines the best of both worlds."

Slide 5: Methodology & Proposed System Architecture

End-to-End Request Pipeline



```
  │              Execution Engines                    │
  │  ┌──────────────────────┬──────────────────────┐  │
  │  │ Deterministic Tools  │ Cloud AI Engine      │  │
  │  │  • Ruff Formatter    │  • Gemini 2.5        │  │
  │  │  • Bandit AST Scan   │    Flash SDK         │  │
  │  │  • Difflib unified   │  • Prompt Guard      │  │
  │  └──────────────────────┴──────────────────────┘  │
  └──────────────────────────────────────────────────┘
                        │
                        ▼
  [ SQLite3 Storage (WAL Mode, Relational Cascades) ]
```

> **Speaker Note:**
> "Every request passes through our rate limiter and security middleware, gets validated against strict Pydantic schemas, checks the SHA-256 cache, and routes either to deterministic local tools or Google Gemini 2.5 Flash."

Slide 6: System Design & Technical Implementation

Key Components & Algorithms

1. **Deterministic Fingerprint Caching:**

$$\text{Key} = \text{SHA256}\Big(\text{JSON_SORT}\big(\{\text{user_id}, \text{tool}, \text{payload}, \text{engine}, \text{model}, \text{prompt_version}\}\big)\Big)$$

   - Guarantees complete cross-tenant cache isolation and instant invalidation on prompt updates.
2. **Cryptographic Authentication Pipeline:**
   - Passwords hashed using **PBKDF2-HMAC-SHA256** with $600{,}000$ iterations and 24-byte salts.
   - Mitigates timing attacks with `hmac.compare_digest` and synthetic dummy hash evaluation.
3. **Concurrency-Safe Persistence (SQLite3 WAL):**
   - Tables: `users`, `sessions`, `history`, `response_cache`, `metrics`, `ai_quota`.
   - Uses `PRAGMA journal_mode=WAL` for non-blocking concurrent reads and atomic writes.
4. **Subprocess Isolation for Local Linters:**
   - Ruff and Bandit are executed in dedicated, time-bounded subprocesses (`timeout=5s` / `timeout=9s`) without network access.

> **Speaker Note:**
> "Our implementation emphasizes software engineering best practices: atomic transactions, sub-process time bounding, and cryptographically verified session storage."

Slide 7: Work Completed So Far (>85% Completed)

Implementation Status & Milestones Achieved

- ☑ **Core Backend Architecture:** FastAPI async application with structured routers, lifecycle handlers, and error boundaries.
- ☑ **All 8 Tools Implemented:**

    - *AI Tools:* Optimise Code, Complexity Analysis, Debug Assistant, Test Case Generator, Code Generator.
    - *Local Tools:* Python Formatter (Ruff), Security Audit (Bandit), Code Comparator (Difflib).
- ✅ **Security & Multi-Tenant Auth:** PBKDF2 credential management, opaque session revocation, CSRF headers, and strict CSP.
- ✅ **Two-Tier Caching System:** Monotonic TTL memory cache + persistent SQLite cache.
- ✅ **Automated Test Suite: 19/19 Unit & Integration Tests Passing** (`tests/test_api.py`, `tests/test_static.py`).
- ✅ **DevOps & Cloud Readiness:** Containerized via `Dockerfile`, Railway blueprint, and Render deployment specifications.

```
====================== 19 passed, 1 warning in 10.21s ======================
All integration tests, authentication tests, and AST scanners verified 100%.
```

> **Speaker Note:**
> "Over 85% of our project work is fully completed, verified, and operational. All 8 intelligence tools are functional, and all 19 unit and integration tests pass cleanly."

Slide 8: Working Project Demonstration & Screenshots

System Screenshots & Live UI Verification

1. **Desktop Workspace UI (`docs/ui-desktop.png`):**
    - Multilingual header (कोडनीती AI), interactive 8-tool selector, syntax-highlighted code editor, line gutter, and live analysis report pane.
2. **Deterministic Diff & Static Formatter:**
    - Real-time side-by-side unified code diff showing line insertions/deletions.
    - Formatted Python snippets generated in $< 25\ \text{ms}$ via Ruff.
3. **AST Security Audit (Bandit):**
    - Identifies high-severity issues (e.g., `subprocess.run` shell-injection) with exact line number references and explanation.
4. **Performance Metrics Dashboard:**
    - Observational latency telemetry displaying cold request times ($\approx 1.2\text{s}$) vs cached responses ($< 5\ \text{ms}$).

(Include screenshots of `docs/ui-desktop.png` and `docs/ui-mobile.png` on this slide)

> **Speaker Note:**
> "Here are direct captures of our working prototype. The UI is completely reactive, built without heavy JS frameworks, and renders reports instantaneously upon cache hits."

Slide 9: Planned Future Work

Roadmap for Remaining 15% & Production Scaling

1. **Distributed Task Queue (Celery + Redis):**

    - Decouple long-running repository-wide static analysis jobs into background workers with Server-Sent Events (SSE) progress streaming.
2. **Multi-Language AST Linting Support:**
    - Expand local offline tooling beyond Python to JavaScript/TypeScript (via Biome/ESLint) and Go (via `gofmt`/`govulncheck`).
3. **Cloud Object Persistence:**
    - Transition SQLite WAL to managed cloud databases (PostgreSQL) and distributed caching (Redis cluster).
4. **Developer IDE Extension:**
    - Package CodeNiti AI REST endpoints into a lightweight **VS Code Extension** for inline code recommendations.

> **Speaker Note:**
> "Our planned future work focuses on horizontal scaling: adding Redis task queues for multi-file repositories, expanding AST linters to JavaScript and Go, and building a VS Code extension."

Slide 10: Signed Project Diary & Review Details

Record of Project Meetings & Guide Consultations

Date	Phase / Agenda	Discussion Points & Guidance Received	Guide Sign
14-08-2026	Problem Definition	Identified LLM code review limitations; Prof. J. N. Mohite suggested hybrid approach with local tools.	*[Signed]*
04-09-2026	Architecture Review	Finalized FastAPI, SQLite WAL mode, and Pydantic validation contracts.	*[Signed]*
25-09-2026	Security & Auth Design	Guide recommended PBKDF2 with $\geq 600{,}000$ iterations and strict zero-execution policy.	*[Signed]*
16-10-2026	Implementation & Caching	Demonstrated L1/L2 cache; verified sub-millisecond cache hit latency.	*[Signed]*
26-10-2026	Review 1 / Progress Check	Validated 19 unit tests, Ruff/Bandit integrations, and live Gemini 2.5 Flash workflow.	*[Signed]*

*Project Diary verified and approved under the guidance of **Prof. J. N. Mohite**.*

> **Speaker Note:**
> "This slide summarizes our regular guide consultations and project diary entries throughout the semester. We thank Prof. J. N. Mohite sir for his valuable guidance. We are now open for questions."